

ft_transcendence 驚き。

概要: 本プロジェクトでは、あなたがまだ一度もコンピュータの世界に足を踏み入れた頃を思い出してほしい。今やあなたはここまで成長したのだ。今こそ輝く時だ！

これまでに行ったことのない課題に取り組むことになる。科学の世界へようこそ。

バージョン: 21.0

I	AI 指示書	2
II	前文	4
II.1	チーム編成とプロジェクト管理	4
	II.1.1 必須のチーム役割	4
	II.1.2 推奨されるプロジェクト管理手法	5
III	必須部分	7
III.1	私たちは何を行うのか？	7
III.2	一般的な要件	8
III.3	技術的要件	9
IV	モジュール	10
IV.1	Web	12
IV.2	アクセシビリティと国際化対応	13
IV.3	ユーザー管理	14
IV.4	人工知能	15
IV.5	サイバーセキュリティ	16
IV.6	ゲーム機能とユーザー体験	16
IV.7	DevOps	18
IV.8	データと分析	19
IV.9	ブロックチェーン	20
IV.10	選択モジュール	20
V	プロジェクトアイデアと事例	21
V.1	事例: 卓球ゲームの開発	21
V.2	ゲーム関連プロジェクト	21
V.3	ソーシャル・共同作業型プロジェクト	22
V.4	クリエイティブ・メディア関連プロジェクト	23
V.5	生産性向上・ツール関連プロジェクト	24
V.6	専門特化型プロジェクト	25
VI	Readme要件	27
VII	ボーナスパート	30
		31
VIII	提出と相互評価	

第I章 AIに関する指示

・ 背景

学習過程において、AIは様々なタスクを支援することができる。AIツールの多様な機能を探求し、それらがどのように作業をサポートできるかを確認する時間を取ろう。ただし、常に慎重にアプローチし、生成された結果を批判的に評価することが重要である。コード、ドキュメント、アイデア、技術的な説明など、どのような内容であっても、自分の質問が適切に形成されているか、生成されたコンテンツが正確であるかを完全に確信することはできない。仲間は、誤りや盲点を回避するための貴重なリソースとなる。

・ 主要なメッセージ

AIを活用して反復的または面倒なタスクを削減する。

コーディングと非コーディングの両方において、将来のキャリアに役立つプロンプト作成スキルを身につける。

AIシステムの仕組みを理解し、一般的リスク、バイアス、倫理的問題をよりの確に予測・回避できるようにする。

仲間と共に作業することで、技術的スキルとリーダーシップスキルの両方をさらに向上させ続ける。

完全に理解し、責任を持てる範囲のAI生成コンテンツのみを使用する。

・ 学習者向けルール:

AIツールを探求し、その仕組みを理解する時間を取ること。これにより、倫理的にツールを活用し、潜在的なバイアスを軽減することができる。

プロンプトを入力する前に問題をよく考察すること。これにより、正確な語彙を用いて、より明確で詳細かつ関連性の高いプロンプトを作成できるようになる。

・ AIが生成したものについては、体系的に確認・検証・疑問視・テストする習慣を身につけるべきである。

・ 常に他者によるレビューを求めること。自身の検証だけに頼らないこと。

・フェーズの成果:

- ・汎用的なプロンプト作成スキルと、特定分野に特化したプロンプト作成スキルの両方を習得する。
- ・AIツールを効果的に活用することで、生産性を向上させる。
- ・計算的思考、問題解決能力、適応力、協働能力の継続的な強化を図る。

・コメントと具体例:

- ・試験や評価など、実際の理解力を問われる場面に定期的に遭遇するだろう。準備を整え、技術的スキルと対人スキルの両方をさらに磨いていくことが重要である。
- ・自分の思考過程を説明したり、仲間と議論したりすることで、理解の不足点が明らかになることが多い。仲間同士の学び合いを積極的に行うようにしよう。
- ・AIツールは多くの場合、あなた特有の文脈を理解できず、一般的な回答を返す傾向がある。同じ環境を共有する仲間は、より適切で正確な洞察を提供してくれるだろう。
- ・AIが最も可能性の高い回答を生成する傾向があるのに対し、仲間は代替的な視点や貴重なニュアンスを提供できる。彼らを品質チェックの手段として活用しよう。

/ 良い実践方法:

私はAIに「ソート関数のテスト方法は？」と尋ねる。するといくつかのアイデアが提示される。私はそれらを試し、結果を仲間とレビューする。私たちは一緒にアプローチを洗練させていく。

X 悪い実践方法:

私はAIに完全な関数を書かせ、それをプロジェクトにコピー&ペーストする。仲間による評価時には、その機能が何をするのか、なぜそうするのかを説明できない。信頼性を失い、プロジェクトも失敗に終わる。

/ 良い実践方法:

私はAIを使ってパーサーの設計を支援する。その後、仲間とロジックについて話し合う。私たちは2つのバグを発見し、一緒に書き直すことで、より優れた、よりクリーンで、完全に理解されたコードを完成させる。

X 悪い実践方法:

私はCopilotにプロジェクトの重要な部分のコードを生成させる。コンパイルは通るものの、パイプ処理の仕組みを説明できない。評価時にその正当性を説明できず、プロジェクトも失敗に終わる。

第II章 前置き

まず第一に、この節目に到達したことを祝福したい！あなたはコモン・コアの最終プロジェクトにいいよ取り組むことになる。そう、これは決して容易なものではない。

超越性（Transcendence）は4～5人で行うグループプロジェクトであり、あなたの創造性、自己信頼、新技術への適応力、そしてチームワークスキルを高めることを目的としている。

チームとして実際のウェブアプリケーションを開発し、選択したモジュールや選択内容に応じて様々な方向に展開できるプロジェクトとなる。作業を開始する前に、チーム全員で十分に検討を重ねることを忘れないように。

本プロジェクトは2つの部分に分かれている：

- ・必須部分：プロジェクトの中核となる固定部分であり、すべてのチームメンバーが貢献しなければならない。
- ・選択モジュール群：自由に選択できるモジュールで、最終成績に反映される。



これは長期にわたるグループプロジェクトである。初期段階での不適切な選択やチーム連携の不足は、多大な時間を浪費することになる。プロジェクト管理とチーム運営は成果に大きく影響する。すべてのチームメンバーが必須部分と選択モジュールの両方に積極的に参加し、貢献する必要がある。

II.1 チーム編成とプロジェクト管理

これはグループプロジェクトであるため、適切なチーム編成が成功の鍵となる。最初から明確な役割分担と責任範囲を定めておく必要がある。

II.1.1 必須のチーム役割

チームは以下の役割を割り当てなければならない（チームメンバーが4人の場合、1人が複数の役割を兼任することも可能である）：

- ・プロダクトオーナー（PO）：製品のビジョンを定義し、機能の優先順位付けを行い、プロジェクトがユーザーニーズを満たすことを保証する。
- ・プロダクトバックログの管理を担当する。

- ・機能の選定や優先順位付けに関する決定を行う。
- ・完了した作業の検証を行う。
- ・関係者（評価者、同僚）との意思疎通を行う。
- ・プロジェクトマネージャー（PM）/スクラムマスター：チームの連携を促進し、障害を取り除く役割を担う。
 - ・チームミーティングや計画会議の運営を行う。
 - ・進捗状況と納期を管理する。
 - ・チーム内のコミュニケーションを確保する。
 - ・リスクや障害を管理する。
- ・技術リード/アーキテクト：技術的な意思決定とアーキテクチャの監督を行う。
 - ・技術的なアーキテクチャを定義する。
 - ・技術スタックに関する決定を行う。
 - ・コード品質とベストプラクティスの遵守を保証する。
 - ・重要なコード変更のレビューを実施する。
- ・開発者（全チームメンバー）：機能やモジュールの実装を担当する。
 - ・割り当てられた機能のコードを記述する。
 - ・コードレビューに参加する。
 - ・自身の実装をテストする。
 - ・作業内容を文書化する。



チーム規模:

- ・4名の場合：一部のメンバーは複数の役割を兼任する場合がある（例：PM兼開発者、PO兼開発者）。
- ・5名の場合：役割をより専門化でき、専任のPO、PM、テックリード、および2名の開発者を配置できる。

すべての役割については、README.mdに明確に記載すること。

II.1.2 推奨されるプロジェクト管理手法

必須ではないが、チームの成功を支援するため、以下の基本的なプロジェクト管理手法の導入を強く推奨する：

- ・定期的なコミュニケーション：進捗確認や障害事項の共有のため、定期的に（週1回または隔週で）ミーティングを実施する。
- ・タスク管理：GitHub Issues、Trello、あるいは共有ドキュメントなどのシンプルなツールを使用して、各メンバーの担当タスクを管理する。

- ・作業の細分化：プロジェクトを小規模で管理可能なタスクに分割する。
- ・コードレビュー：重要なコード変更については、少なくとも他のチームメンバー1名によるレビューを実施するよう努める。
- ・文書化：重要な決定事項やシステムの動作方法についての記録を保管する。
- ・コミュニケーションチャンネル：DiscordやSlackなどのツールを活用し、チーム内の迅速なコミュニケーションを図る。



1これらの実践方法は、作業を効率的に整理するための推奨事項である。チームに最適な方法を見つけよう！重要なのは、全員が積極的に関与し、作業が適切に調整されることである。



評価段階では、チームは以下の点について説明を求められる：

- ・役割分担の方法
- ・作業の組織化と分担方法
- ・チームとしてのコミュニケーションと調整方法
- ・各メンバーのプロジェクトへの貢献内容

すべてのチームメンバーは、プロジェクト全体とその各自の貢献について説明できる必要がある。

第III章 必須部分

プロジェクトの内容は自由に設定できる。そう、あなた自身のアイデアを持ち寄り、チームとしてどのようなアプリケーションを開発するかを共に決定しなければならない。

これは共通コアで以前に取り組んだ内容からさらに発展したものである。単に実装する機能だけでなく、プロジェクト全体を包括的に考える必要がある。

もちろん、完全に自由というわけではなく、一定の制約は存在するが、そのアイデア自体はあなたのものである。

III.1 私たちは何をしているのか?

まず初めに、包括的なREADME.mdファイルを作成する必要がある。READMEの詳細な要件については、本文書末尾の「README要件」セクションに記載されている。



プロジェクト例: あなたのプロジェクトは様々な形態を取り得る。以下にいくつかの有効な例を示す:

- ・ トーナメントシステムを備えたマルチプレイヤーPongゲーム
- ・ リアルタイム機能を備えた共同作業プラットフォーム
- ・ ユーザー間のインタラクションが可能なソーシャルネットワーク
- ・ マッチング機能を備えたオンラインゲーム（チェス、三目並べなど）
- ・ プロジェクト管理アプリケーション
- ・ その他、要件を満たす創造的なウェブアプリケーション

重要なのは、技術的なスキルと創造性を発揮できる、魅力的なプロジェクトを作成することである。

III.2 一般要件

プロジェクト全体を一から構築するのは複雑で、多くの問題が発生する可能性がある。支援のため、遵守すべき一般要件のリストを提供する。これらの要件に従わない場合、プロジェクトは却下される。

要件は以下の通りである：

- ・プロジェクトはウェブアプリケーション形式で、フロントエンド、バックエンド、およびデータベースを必要とすること。
- ・Gitを使用し、明確で意味のあるコミットメッセージを付けること。リポジトリには以下の内容が示されている必要がある：
 - すべてのチームメンバーによるコミット履歴
 - 変更内容を明確かつ簡潔に記述したコミットメッセージ
 - チーム全体での適切な作業分担状況
- ・デプロイメントにはコンテナ化ソリューション（Docker、Podman、または同等のもの）を使用し、単一のコマンドで実行可能であること。
- ・あなたのウェブサイトはGoogle Chromeの最新安定バージョンと互換性がある状態であること。
- ・ブラウザのコンソールに警告やエラーメッセージが表示されてはならない。
- ・プロジェクトには、関連する内容を含むアクセシブルなプライバシーポリシーおよび利用規約ページを必ず含めること。



プライバシーポリシーおよび利用規約: これらのページは評価時に検証対象となる。以下の要件を満たしている必要がある:

- ・アプリケーションから容易にアクセス可能であること（例：フッターリンクなど）
- ・プロジェクトに関連する適切な内容を記載していること。

単なるプレースホルダーや空のページであってはならない。

プライバシーポリシー/利用規約ページが欠落している場合や内容が不十分な場合、プロジェクトは却下される。



マルチユーザー対応（必須）: あなたのウェブサイトは複数のユーザーが同時に利用できるような設計されている必要がある。これはプロジェクトの中核要件である。ユーザーは競合やパフォーマンスの問題なく、同時にアプリケーションを操作できる状態でなければならない。具体的には以下の要件を満たす必要がある:

- .. 複数のユーザーが同時にログインし、アクティブな状態を維持できる。
- ・異なるユーザーによる同時操作が適切に処理される。
- ・該当する場合、リアルタイムの更新がすべての接続ユーザーに即座に反映される。
- ・同時操作によるデータの破損や競合状態が発生しない。

III.3 技術要件

このセクションは前のセクションと同様、必須要件である。次章では、使用するモジュールを選択できるようになる。

\\· すべてのデバイスで明確かつ応答性が高く、アクセシビリティに優れたフロントエンド。

\\· 任意のCSSフレームワークまたはスタイリングソリューションを使用する（例：Tailwind CSS、Bootstrap、Material-UI、Styled Componentsなど）。

\\· 認証情報（APIキー、環境変数など）をGitで無視されるローカルの.envファイルに保存し、.env.exampleファイルを提供する。

\\ \\ · データベースには明確なスキーマと明確に定義されたリレーションが必要である。

\\ \\ \\ · アプリケーションには基本的なユーザー管理システムが実装されている必要がある。ユーザーは安全に登録およびログインできる機能を備えていること：

○ 最低限、適切なセキュリティを備えたメールとパスワードによる認証（ハッシュ化されたパスワード、ソルト処理など）を実装すること。

\\ \\ \\ · 追加の認証方法（OAuth、2段階認証など）は、モジュールを介して実装可能である。

\\ \\ \\ · すべてのフォームおよびユーザー入力、フロントエンドとバックエンドの両方で適切に検証されなければならない。

\\ \\ \\ · バックエンドでは、すべての通信においてHTTPSを使用すること。

フレームワークとは何か？ 本プロジェクトにおいて、フレームワークとは以下のような包括的なツールを指す：

\\ \\ \\ · コードを整理するための体系的なアーキテクチャと規約

\\ \\ \\ · 一般的なタスクに対する組み込み機能（ルーティング、状態管理など）

\\ \\ \\ · 各種ツールやライブラリからなる完全なエコシステム

\\ \\ \\ 具体例：



\\ \\ \\ · フロントエンドフレームワーク：React、Vue、Angular、Svelte、Next.js（これらはフレームワークに該当する）

\\ \\ \\ · バックエンドフレームワーク：Express、Fastify、NestJS、Django、Flask、Ruby on Rails

\\ \\ \\ · フレームワークではないもの：jQuery（ライブラリ）、Lodash（ユーティリティライブラリ）、Axios（HTTPクライアント）

\\ \\ \\ 注：Reactは技術的にはライブラリに分類されるが、エコシステムとアーキテクチャパターンの観点から、この文脈ではフレームワークと見なされる。

第IV章 モジュール

プロジェクトを完了するためには、合計で14ポイントを獲得する必要がある。各主要モジュールは2ポイント、各副次モジュールは1ポイントの価値がある。

\\\\\\以下のカテゴリが用意されている。どのカテゴリからでも複数のモジュールを選択することが可能である：

- | | |
|----------------|----------------------|
| ・ Web | ・ ゲームおよびユーザーエクスペリエンス |
| ・ アクセシビリティと国際化 | ・ DevOps |
| ・ ユーザー管理 | ・ データと分析 |
| ・ 人工知能 | ・ ブロックチェーン |
| ・ サイバーセキュリティ | ・ 選択モジュール |

アイデアが明確になり、構築したいものについて十分な理解を得た後にモジュールを選択することを強く推奨する。

さらに、評価期間中に一部のモジュールが検証されない場合に備えて、合計14ポイント以上を目指すのが賢明な選択となる場合がある。

重要 - モジュールの依存関係と評価について:



- ・一部のモジュールでは、他のモジュールを先に実装する必要がある場合がある（情報注釈で示されている）。
- ・ゲーム関連モジュール（AI対戦相手、トーナメント、ゲームカスタマイズ、観戦モード、マルチプレイヤー3人以上、別のゲームを追加）については、少なくとも1つのゲームを先に実装しておく必要がある。
- ・ゲーム統計モジュールでは、ゲームが実装されていることが必須条件となる。
- ・高度なチャット機能を使用するには、「ユーザーインタラクション」モジュールの基本チャット機能が実装されている必要がある。
- ・SSRはICPブロックチェーンバックエンドとは互換性がない。
- ・モジュール間の連携が円滑に行えるよう、モジュールの設計は慎重に行うこと！

評価時の注意: 各実装済みモジュールについてデモンストレーションが求められる。完全に機能し適切に実装されたモジュールのみが最終評価点に反映される。機能しないモジュールや実装が不十分なモジュールは0点となる。

IV.1 Web

主要事項: フロントエンドとバックエンドの両方にフレームワークを使用すること。

- フロントエンドフレームワークを使用する (React、Vue、Angular、Svelteなど)。
- バックエンドフレームワークを使用する (Express、NestJS、Django、Flask、Ruby on Railsなど)。
- フロントエンドとバックエンドの両方の機能を備えたフルスタックフレームワーク (Next.js、Nuxt.js、SvelteKitなど) を使用する場合、両方の機能をカウントする。

・副次事項: フロントエンドフレームワークを使用する (React、Vue、Angular、Svelteなど)。

- 副次事項: バックエンドフレームワークを使用する (Express、Fastify、NestJS、Djangoなど)。

- 主要事項: WebSocketなどの技術を使用してリアルタイム機能を実装する。

- クライアント間でのリアルタイム更新を実現する。
- 接続/切断を適切に処理すること。
- 効率的なメッセージブロードキャストを行うこと。
- 主要事項: ユーザーが他のユーザーと相互にやり取りできるようにする。最低限必要な要件は以下の通りである:
 - 基本的なチャットシステム (ユーザー間でのメッセージの送受信機能) を実装すること。
 - プロフィールシステム (ユーザー情報の閲覧機能) を実装すること。
 - フレンドシステム (フレンドの追加/削除機能、フレンドリストの表示機能) を実装すること。
- 主要事項: セキュアなAPIキーによるデータベース操作、レート制限、ドキュメント提供、および少なくとも5つのエンドポイントを備えた公開APIを実装すること:

- GET /api/{何か}
- POST /api/{何か}
- PUT /api/{何か}
- DELETE /api/{何か}
- 副次事項: データベース操作にはORMを使用すること。
- 副次事項: すべての作成、更新、削除操作acに対応した完全な通知システムを実装すること。
- 副次事項: リアルタイムの共同作業機能 (共有ワークスペース、ライブ編集、共同描画機能など) を実装すること。
- 副次事項: パフォーマンス向上とSEO最適化のため、サーバーサイドレンダリング (SSR) を実装すること。
- 副次事項: オフライン対応とインストール可能なプログレッシブウェブアプリ (PWA) を実装すること。
- 副次事項: 適切なカラーパレット、タイポグラフィ、アイコンを含む再利用可能なコンポーネントで構成されたカスタムデザインシステムを構築すること (最低10個の再利用可能なコンポーネントを含む)。

- 副次事項: フィルタリング、ソート、ページネーション機能を備えた高度な検索機能を実装すること。
- 副次事項: ファイルのアップロードおよび管理システムを実装すること。
 - 複数のファイル形式（画像、文書など）をサポートすること。
 - クライアントサイドおよびサーバーサイドでの検証機能（ファイルタイプ、サイズ、フォーマット）を実装すること。
 - 適切なアクセス制御を備えた安全なファイルストレージを実装すること。
 - 該当する場合、ファイルプレビュー機能を実装すること。
 - アップロードの進捗状況を表示する機能を実装すること。
 - o アップロードしたファイルを削除する機能を備えること。

IV.2 アクセシビリティと国際化

- 主要機能: スクリーンリーダー対応、キーボードナビゲーション、支援技術の利用など、WCA G 2.1 AA基準に準拠した完全なアクセシビリティを確保すること。
- 副次機能: 複数言語（少なくとも3言語）に対応すること。
 - i18n（国際化）システムを実装すること。
 - 少なくとも3言語の完全な翻訳を実装すること。
 - UI内に言語切り替え機能を設けること。
 - すべてのユーザー向けテキストは翻訳可能である必要がある。
- 副次機能: 右から左へ記述する言語（RTL言語）への対応。
 - 少なくとも1つのRTL言語（アラビア語、ヘブライ語など）をサポートすること。
 - レイアウト全体を鏡像化すること（テキストの方向だけでなく）。
 - RTL言語特有のUI調整が必要な場合に対応すること。
 - o 左から右への記述言語（LTR）と右から左への記述言語（RTL）のシームレスな切り替えが可能であること。
- 副次機能: 追加のブラウザへの対応。
 - 少なくとも2つの追加ブラウザ（Firefox、Safari、Edgeなど）との完全な互換性を確保すること。
 - o 各ブラウザにおいて全ての機能をテストし、不具合を修正すること。
 - o ブラウザ固有の制限事項があれば文書化すること。
 - サポート対象の全ブラウザにおいてUI/UXの表示を統一すること。

IV.3 ユーザー管理

- 主要機能: 標準的なユーザー管理および認証機能。
 - ユーザーが自身のプロフィール情報を更新できるようにすること。
 - ユーザーがアバターをアップロードできるようにすること（アバターが提供されない場合はデフォルトのアバターを使用する）。
 - o ユーザーは他のユーザーを友達として追加でき、そのオンライン状態を確認できるようにすること。
 - o ユーザーには自身の情報を表示するプロフィールページを設けること。
- 未成年者向け: ゲームの統計情報および対戦履歴（ゲームモジュールが必要）。
 - ユーザーのゲーム統計情報（勝利数、敗北数、ランキング、レベルなど）を追跡すること。
 - 対戦履歴を表示すること（1対1のゲーム、日付、結果、対戦相手など）。
 - o 達成項目や進捗状況を表示すること。
 - o リーダーボードとの連携機能。

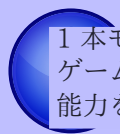


本モジュールを使用するには、少なくとも1つのゲームを実装している必要がある（「ゲームとユーザー体験」セクション参照）。機能的なゲームが実装されていない場合、このモジュールの認定を受けることはできない。

- 軽微な要件: OAuth 2.0を使用したりリモート認証を実装すること（Google、GitHub、42など）。
- 主要な要件: 高度な権限管理システムを実装すること:
 - ユーザーの閲覧、編集、削除（CRUD操作）が可能であること。
 - 役割管理機能を実装すること（管理者、一般ユーザー、ゲスト、モデレーターなど）。
 - o ユーザーの役割に応じて異なる表示内容や操作権限を設定すること。
- 主要な要件: 組織管理システムを実装すること:
 - 組織の作成、編集、削除が可能であること。
 - ユーザーに組織を割り当てる機能を実装すること。
 - ユーザーを組織から解除する機能を実装すること。
 - 組織の閲覧機能を実装し、ユーザーが組織内で特定の操作（最低限: 作成、読み取り、更新）を実行できるようにすること。
- 副次的な要件: ユーザーに対して完全な2段階認証（Two-Factor Authentication）システムを実装すること。
- 副次的な要件: ユーザーアクティビティの分析およびインサイトダッシュボードを実装すること。

IV.4 人工知能

- 主要な要件: ゲーム用のAI対戦相手を導入すること。
 - o AIは挑戦的であり、時折勝利できる能力を備えている必要がある。
 - o AIは人間らしい行動を模倣すべきである（完璧なプレイではなく）。
 - o ゲームのカスタマイズオプションを実装する場合、AIはそれらの機能を使用できるようにしなければならない。
 - o 評価時には、自分のAI実装について明確に説明できる必要がある。



1 本モジュールでは、少なくとも1つのゲームを実装済みであることが求められる（「ゲームとユーザー体験」セクション参照）。AIはあなたのゲームを適切にプレイできる能力を備えていなければならない。

- ・ 主要課題: RAG（Retrieval-Augmented Generation）システムを完全に実装すること。
 - o 大規模な情報データセットと相互作用すること。
 - o ユーザーが質問をすることができ、関連する回答が得られること。
 - ・ 適切な文脈情報の検索と応答生成を実装すること。
- ・ 主要課題: LLM（Large Language Model）システムインターフェースを完全に実装すること。
 - o ユーザー入力に基づいてテキストおよび/または画像を生成すること。
 - ・ ストリーミング応答を適切に処理すること。
 - o エラー処理とレート制限を実装すること。
- ・ 主要課題: 機械学習を用いた推薦システムを実装すること。
 - o ユーザーの行動履歴に基づいたパーソナライズド推薦を行うこと。
 - ・ 協調フィルタリングまたはコンテンツベースフィルタリングを採用すること。
 - o 時間の経過とともに推薦内容を継続的に改善すること。
- ・ 副次課題: コンテンツモデレーション AI（自動モデレーション、自動削除、自動警告など）
- ・ 副次課題: アクセシビリティ向上やインタラクションのための音声/音声認識機能の統合。
- ・ 副次課題: ユーザー生成コンテンツに対する感情分析。
- ・ 副次課題: 画像認識およびタグ付けシステム。

IV.5 サイバーセキュリティ

- ・ 主要課題: WAF/ModSecurity（強化版）の実装 + 秘密情報管理のためのHashiCorp Vault:
 - ・ ModSecurity/WAFの厳格な設定。
 - ・ Vaultでの秘密情報管理（APIキー、認証情報、環境変数など）を暗号化・隔離して行う。

IV.6 ゲームおよびユーザー体験

- ・ 主要課題: ユーザーが互いに対戦可能な完全なWebベースゲームを実装する。
 - ゲームはリアルタイムマルチプレイヤー形式とすることができる（例：ピンポン、チェス、三目並べ、カードゲームなど）。
 - ・ プレイヤーはライブ対戦を行えるようにしなければならない。
 - ゲームには明確なルールと勝敗条件が設定されている必要がある。
 - ・ ゲームは2Dまたは3D形式で実装可能である。
 - ・ 主要課題: 遠隔プレイヤー対応 - 別々のコンピュータを使用している2人のプレイヤーがリアルタイムで同じゲームをプレイできるようにする。
 - ネットワーク遅延や切断が発生した場合でも適切に処理すること。
 - 遠隔プレイにおいてもスムーズなユーザー体験を提供すること。
 - ・ 再接続ロジックを実装すること。
- ・ 主要課題: マルチプレイヤーゲーム（2人以上のプレイヤー対応）。
 - ・ 3人以上のプレイヤーが同時に参加できる機能を実装すること。
 - ・ すべての参加者にとって公平なゲームプレイメカニズムを確保すること。
 - ・ すべてのクライアント間で適切な同期を行うこと。



1 本モジュールでは、少なくとも1つのゲームを実装済みであることが求められる（「ゲームとユーザー体験」セクション参照）。本課題では、ゲームを拡張して3人以上のプレイヤー対応を実現すること。

- ・ 主要機能: ユーザー履歴とマッチング機能を備えた別のゲームを追加すること。
 - 第2の独立したゲームを実装すること。
 - 当該ゲームのユーザー履歴と統計情報を追跡すること。
 - ・ マッチングシステムを実装すること。
 - ・ パフォーマンスと応答性を維持すること。

i 本モジュールでは、既に最初のゲームを実装済みであることが求められる（上記「完全なWebベースゲームの実装」モジュール参照）。機能的な最初のゲームが存在しない場合、本モジュールの履修は認められない。

・主要課題: Three.jsやBabylon.jsなどのライブラリを使用して高度な3Dグラフィックスを実装すること。

- ・没入感のある3D環境を構築すること。
- o 高度なレンダリング技術を実装すること。
- ・スムーズな動作とユーザーインタラクションを確保すること。

・副次課題: 高度なチャット機能の実装（「ユーザーインタラクション」モジュールで扱った基本チャット機能を拡張するもの）。

- o ユーザーからのメッセージ受信をブロックする機能。
- ・チャットから直接ゲームをプレイできるようにユーザーを招待する機能。
- ・チャット内でのゲーム/トーナメントの通知機能。
- o チャットインターフェースからユーザープロフィールにアクセスできる機能。
- o チャット履歴の保持機能。
- ・入力中インジケータと既読確認機能。

i 本モジュールは「ユーザー相互作用を許可する」モジュールで実装した基本チャットシステムを拡張するものである。基本チャット機能を実装していない場合、本モジュールの実装を主張することはできない。

・軽微な機能追加: トーナメントシステムを実装する。

- ・明確な対戦順序とトーナメント表システム。
- o 対戦順序を明確に表示し、トーナメント表を管理する。 o 誰と誰が対戦するかを追跡する。
- o トーナメント参加者のためのマッチングシステム。
- o トーナメントの登録および管理機能。

i 本モジュールを実装するには、少なくとも1つのゲームを実装している必要がある（「ゲームとユーザー体験」セクション参照）。ゲームが存在しない場合、トーナメントを開催することは不可能である。

・軽微な機能: ゲームのカスタマイズオプション。

- o パワーアップ、攻撃、または特殊能力などの追加機能。

- 異なるマップやテーマ。
- ・ カスタマイズ可能なゲーム設定。
- デフォルト設定は必ず利用可能であること。



1 本モジュールを実装するには、少なくとも1つのゲームを実装している必要がある（「ゲームとユーザー体験」セクション参照）。既存のゲームにカスタマイズ機能を追加するものである。

- ・ 軽微な機能: ユーザーの行動に対して報酬を与えるゲーミフィケーションシステム。
 - ・ 以下のうち少なくとも3つを実装すること: 実績システム、バッジ、リーダーボード、経験値/レベルシステム、デイリーチャレンジ、報酬
 - ・ システムは永続的である必要があり（データベースに保存される）、
 - ユーザーに対する視覚的フィードバック（通知、進捗バーなど）を提供すること
 - ・ 明確なルールと進行メカニズムを備えていること



本モジュールは軽微な機能（1ポイント）であるが、完全なゲーミフィケーションシステムを実装することは大きな作業となる。量より質を重視し、6つの機能を中途半端に実装するよりも、3つの機能を高品質に実装することが望ましい。

- ・ 軽微な機能: ゲームの観戦モードを実装する。
 - ユーザーが進行中のゲームを観戦できるようにすること。
 - ・ 観戦者向けのリアルタイム更新機能。
 - オプション機能: 観戦者間のチャット機能。



1 本モジュールでは、少なくとも1つのゲームを実装済みであることが求められる（「ゲームとユーザー体験」セクション参照）。観戦者には観戦対象のゲームが必要である。

IV.7 Devops

- ・ 主要な機能: ELKスタック（Elasticsearch、Logstash、Kibana）を用いたlog管理用インフラストラクチャの構築。
 - Elasticsearchはログの保存とインデックス作成に使用。
 - ・ Logstashはログの収集と変換に使用。
 - Kibanaは可視化およびダッシュボード作成に使用。

(※原文の"Surprise."は具体的な内容が不明なため、そのまま翻訳) - ログ保持ポリシーとアーカイブポリシーを実装すること。

- すべてのコンポーネントに対する安全なアクセスを確保すること。
- ・ 主要な機能: PrometheusとGrafanaを使用した監視システム。
- ・ Prometheusを設定してメトリクスを収集すること。
- ・ エクスポーターと統合機能を設定すること。
- Grafanaでカスタムダッシュボードを作成すること。
- ・ アラートルールを設定すること。 o Grafanaへのアクセスを安全に管理すること。
- ・ 主要な機能: マイクロサービスアーキテクチャによるバックエンド。
- ・ 明確なインターフェースを備えた疎結合なサービスを設計すること。 o 通信にはREST APIまたはメッセージキューを使用すること。
- o 各サービスは単一の責務を持つべきであること。
- ・ 副次的な機能: 自動復旧手順を備えたヘルスチェックおよびステータスページシステム。

バックアップおよび災害

IV.8 データと分析

- ・ 主要な機能: データ可視化機能を備えた高度な分析ダッシュボード。 ion.
- ・ インタラクティブなチャートおよびグラフ（折れ線グラフ、棒グラフ、円グラフなど）。
- o リアルタイムデータの更新機能。
- ・ エクスポート機能（PDF、CSVなど）。
- ・ カスタマイズ可能な日付範囲およびフィルタ機能。
- ・ 副次的な機能: データのエクスポートおよびインポート機能。
- ・ 複数の形式でデータをエクスポート可能（JSON、CSV、XML、etc.）。
- ・ 検証機能を備えたデータインポート機能。etc.）。
- ・ 一括操作機能をサポート。
- ・ 副次的な機能: GDPR対応機能。
- o ユーザーが自身のデータの開示を請求できるようにする。
- o 確認を伴うデータ削除機能。
- ・ ユーザーが読みやすい形式で自身のデータをエクスポート可能。
- ・ データ操作に関する確認メールの送信機能。

IV.9 ブロックチェーン

- ・ 主要な機能: トーナメントのスコアをブロックチェーンに記録する。
 - AvalancheとSolidityスマートコントラクトをテスト用ブロックチェーン上で使用。
- ・ トーナメントのスコアを記録・管理・取得するためのスマートコントラクトを実装。
- ・ データの完全性と不変性を確保。
- ・ 副次的な機能: ICP（Internet Computer Protocol）をブロックチェーン上で動作するバックエンドとして使用（SSRとは互換性なし）。

IV.10 選択モジュール

- ・ 主要機能: 上記リストに記載されていない独自のモジュールを実装。
 - そのモジュールは実質的であり、技術的に高度な内容を示すものでなければならない。
- ・ README.mdにおいて、以下の点について適切な説明を提供すること:
 - * 当該モジュールを選択した理由
 - * それが解決する技術的課題
 - * プロジェクトにどのような付加価値をもたらすか
 - * なぜ「主要」モジュールとしての評価に値するか（2ポイント）
- ・ 安易な方法を用いたり、些細な機能を実装した場合、審査は不合格となる。
 - 創造的に考え、既成概念にとらわれない発想をすること。
 - 当該モジュールはプロジェクトの文脈に関連している必要がある。
- ・ 「副次的」モジュール: 「主要」モジュールと同様だが、範囲が限定的で複雑さが少ない。
 - ・ 技術的スキルと創造性を依然として示す必要がある。
 - ・ プロジェクトに対して有意義な価値を加えるものでなければならない。
 - ・ README.mdでの正当性説明が必要（「主要」モジュールと同様だが、評価ポイントは1点）。

第V章 プロジェクトアイデアと事例

プロジェクト開始の手助けとし、創造性を刺激するため、本章では必要な14ポイントを達成するための具体的なプロジェクトアイデアと事例を紹介する。これらは単に提案例に過ぎないため、自由に創造的に考え、独自のユニークなアイデアを生み出してほしい！

V.1 事例: ピンポンゲームの開発

もしピンポンゲーム（オリジナルプロジェクトと同様のもの）を作成する場合、14ポイントを達成するための方法は以下の通りである：

- ・ゲーム性とユーザー体験：Webベースのゲーム（2ポイント） + 遠隔プレイヤー対応機能（2ポイント） + トーナメントシステム実装（1ポイント） + ゲームカスタマイズ機能（1pt） = 6 ポイント
- ・ユーザー管理：標準的なユーザー管理システム（2ポイント） + OAuth認証（1pt） = 3 ポイント
- ・Web技術：フレームワークの活用（フロントエンド + バックエンド = 2ポイント） + ORM実装（1pt） = 3 ポイント
- ・AI技術：AI対戦機能（2pts） = 2 ポイント

合計: 14ポイント



これはあくまで一例である。異なるカテゴリのモジュールを自由に組み合わせることで、独自のプロジェクトを作成することが可能だ。重要なのは、各モジュールが相互に連携し、アプリケーションに付加価値をもたらすように設計することである。

V.2 ゲーム関連プロジェクト

これらのプロジェクトはインタラクティブなゲームプレイとユーザー間の競争に焦点を当てている：

- ・マルチプレイヤー・ピンポン：トーナメント形式、遠隔プレイ、AI対戦相手、パワーアップ機能を備えた古典的なピンポンゲーム。

推奨モジュール：Webベースゲーム、遠隔プレイヤー機能、トーナメントシステム、AI対戦相手、ゲームカスタマイズ機能

ポイント可能性：14ポイント以上

・オンラインチェスプラットフォーム：マッチング機能、ELOレーティングシステム、ゲーム分析機能、観戦モードを備えたリアルタイムチェス。

○ 推奨モジュール：Webベースゲーム、遠隔プレイヤー機能、AI対戦相手、観戦モード、ゲーム統計機能

・ ポイント可能性：15ポイント以上

・ カードゲームアリーナ：ポーカーやウノなどのマルチプレイヤーカードゲームをトーナメント形式で提供し、リーダーボード機能を備えたプラットフォーム。

○ 推奨モジュール：Webベースゲーム、マルチプレイヤー3+トーナメントシステム、ゲーミフィケーション機能

○ ポイント可能性：14ポイント以上

・ バトルロイヤルミニゲーム：複数プレイヤー対応のシンプルなブラウザベースのバトルロイヤルゲーム。

○ 推奨モジュール：Webベースゲーム、マルチプレイヤー3人以上、リアルタイム機能、ゲームカスタマイズ機能

・ ポイント可能性：14ポイント以上

・ トリビア/クイズプラットフォーム：カテゴリ別・トーナメント形式のリアルタイムマルチプレイヤークイズゲーム。

・ 推奨モジュール：Webベースゲーム、マルチプレイヤー3+トーナメントシステム、ゲーミフィケーション、分析ダッシュボード

○ ポイント可能性：15ポイント以上

V.3 ソーシャル・共同プロジェクト

これらのプロジェクトはユーザー間のインタラクションとコミュニティ形成を重視している：

・ ソーシャルネットワーク：ユーザープロフィール、投稿、コメント、いいね、友達機能、リアルタイムチャット、通知機能。

○ 推奨モジュール：ユーザーインタラクション、リアルタイム機能、通知システム、高度なチャット、ファイルアップロード

・ ポイントポテンシャル：14ポイント以上

・ 共同作業スペース：リアルタイム文書編集、プロジェクト管理、チームチャット、ファイル共有。

○ 推奨モジュール：リアルタイム共同作業機能、ユーザーインタラクション、組織管理システム、ファイルアップロード、高度な権限管理

○ ポイントポテンシャル：15ポイント以上

・ フォーラムプラットフォーム：カテゴリ別掲示板、スレッド機能、モデレーションツール、ユーザー評価システムを備えたディスカッションボード。

- ・推奨モジュール：ユーザーインタラクション、高度な権限管理、ゲーミフィケーション、コンテンツモデレーションAI、高度な検索機能
- ポイントポテンシャル：14ポイント以上
- ・イベント管理プラットフォーム：イベントの作成・管理、参加受付システム、カレンダー連携、通知機能を備えたプラットフォーム。
- ・推奨モジュール：ユーザーインタラクション、通知システム、組織管理システム、公開API、高度な検索機能
- ポイントポテンシャル：14ポイント以上
- ・学習管理システム：コース、課題、小テスト、学習進捗管理、教員・学生間のインタラクション機能を備えたシステム。
- ・推奨モジュール：ユーザーインタラクション、組織管理システム、高度な権限管理、ファイルアップロード、分析ダッシュボード
- ポイントポテンシャル：15ポイント以上

V.4 クリエイティブ&メディアプロジェクト

これらのプロジェクトはコンテンツの作成と共有に焦点を当てたものである：

- ・音楽ストリーミングプラットフォーム：音楽やプレイリストのアップロード・ストリーミング、おすすめ機能、ソーシャル機能を備えたプラットフォーム。
- ・推奨モジュール：ファイルアップロード、ユーザーインタラクション、レコメンデーションシステム、高度な検索、分析ダッシュボード
- ・潜在的なポイント：15ポイント以上
- ・動画共有プラットフォーム：動画のアップロード・視聴、コメント、いいね、サブスクリプション、おすすめ機能を備えたプラットフォーム。
- ・推奨モジュール：ファイルアップロード、ユーザーインタラクション、レコメンデーションシステム、コンテンツモデレーションAI、高度な検索
- ・潜在的なポイント：16ポイント以上
- ・アートギャラリー：ギャラリーで作品を公開し、コメント、いいね、アーティストプロフィールを管理できるプラットフォーム。
- ・推奨モジュール：ファイルアップロード、ユーザーインタラクション、画像認識、高度な検索、カスタムデザインシステム
- 潜在的なポイント：14ポイント以上
- ・ブログプラットフォーム：ブログの作成・公開が可能で、コメント、タグ、カテゴリ、読者エンゲージメント機能を備える。
- ・推奨モジュール：ユーザーインタラクション、サーバーサイドレンダリング、高度な検索、感情分析、多言語対応
- 潜在的なポイント：14ポイント以上

・レシピ共有プラットフォーム：レシピの共有、評価・コメント機能、献立作成、買い物リスト管理が可能。

推奨モジュール：ユーザーインタラクション、ファイルアップロード、高度な検索、推薦システム、PWA

・潜在的なポイント：14ポイント以上

V.5 生産性向上・ツール系プロジェクト

これらのプロジェクトはユーザーが業務を整理・管理する上で役立つ：

・タスク管理システム：プロジェクト、タスク、課題、期限、チームコラボレーション、進捗管理が可能。

・推奨モジュール：組織管理システム、ユーザーインタラクション、リアルタイム共同作業機能、通知システム、分析ダッシュボード

○ 潜在的なポイント：15ポイント以上

・コード共同作業プラットフォーム：コードスニペットの共有、共同コーディング、バージョン管理、ディスカッション機能を備える。

○ 推奨モジュール：ユーザーインタラクション、リアルタイム共同作業機能、公開API、高度な検索機能、カスタムデザインシステム

○ 潜在的なポイント：14ポイント以上

・予約システム：リソース（会議室、機器、アポイントメント）の予約、カレンダー、通知機能を提供する。

・推奨モジュール：ユーザーインタラクション、組織管理システム、通知システム、公開API、高度な検索機能

○ 潜在的なポイント：14ポイント以上

・マーケットプレイスプラットフォーム：物品の売買機能、ユーザー評価システム、メッセージング機能、決済統合、検索機能を備える。

・推奨モジュール：ユーザーインタラクション、ファイルアップロード、高度な検索機能、推薦システム、公開API

・潜在的なポイント：14ポイント以上

・フィットネストラッカー：トレーニング記録、進捗管理、チャレンジ機能、リーダーボード、ソーシャル機能を備える。

○ 推奨モジュール：ユーザーインタラクション、ゲーミフィケーション、分析ダッシュボード、PWA、データエクスポート/インポート

○ 潜在的なポイント：14ポイント以上

V.6 専門プロジェクト

これらのプロジェクトは特定のニッチ分野や業界を対象とする：

- ・リアルタイム取引シミュレーター：リアルタイムデータとポートフォリオを使用した株式および仮想通貨取引のシミュレーション。

- o 推奨モジュール：リアルタイム機能、ユーザーインタラクション、分析ダッシュボード、公開API、高度な3Dグラフィックス

- ・潜在的なポイント：15ポイント以上

- ・語学学習プラットフォーム：レッスン、演習、進捗管理、相互練習機能を備えた学習プラットフォーム。

- o 推奨モジュール：リアルタイム機能、ユーザーインタラクション、ゲーミフィケーション、多言語対応、音声機能統合、分析ダッシュボード

- o 潜在的なポイント：15ポイント以上

- ・ペット里親募集プラットフォーム：ペットの検索、里親手続き、ユーザープロフィール、メッセージ機能を備えたプラットフォーム。

- ・推奨モジュール：ユーザーインタラクション、ファイルアップロード、高度な検索機能、組織管理システム、通知システム

- ・潜在的なポイント：14ポイント以上

- ・旅行計画プラットフォーム：旅行プランの作成、旅程の共有、おすすめ情報、ソーシャル機能を備えたプラットフォーム。

- ・推奨モジュール：ユーザーインタラクション、リアルタイム共同編集機能、推薦システム、多言語対応、高度な検索機能

- ・潜在的なポイント：15ポイント以上

- ・クラウドファンディングプラットフォーム：キャンペーンの作成、寄付受付、最新情報の提供、コミュニティエンゲージメントを実現するプラットフォーム。

- 推奨モジュール：ユーザーインタラクション、ファイルアップロード機能、公開API、分析ダッシュボード、通知システム

- 潜在的なポイント：14ポイント以上

これらは単にアイデアを提供するものであり、重要なのは以下の条件を満たすプロジェクトを選択することである：



- ・チームの関心を引き、全員が意欲的に取り組めるプロジェクトであること。
- ・必要なモジュール（最低14ポイント）を実装可能なプロジェクトであること。
- ・技術的複雑性と創造性を実証できるプロジェクトであること。
- ・プロジェクトのスケジュール内で現実的に完了可能なプロジェクトであること。
- ・モジュール間の組み合わせが論理的に整合性があり、互いにうまく機能するプロジェクトであること。

チームと協議し、利用可能なモジュールを検討した上で、慎重に選択すること！

第VI章 README要件

Gitリポジトリのルートディレクトリには、README.mdファイルを必ず作成すること。このファイルの目的は、プロジェクトに不慣れな関係者（同僚、スタッフ、採用担当者など）が、プロジェクトの内容、実行方法、および関連情報の入手先を迅速に理解できるようにすることである。

README.mdには以下の項目を必ず含めること：

- ・最初の行は斜体とし、「本プロジェクトは<login1>[、<login2>[、<login3> [...] /j
- ・A『説明』セクションを設け、プロジェクトの目的と概要を明確に記載すること。
- ・An『手順』セクションには、コンパイル、インストール、および実行に関する必要な情報をすべて記載すること。
- ・「参考資料」セクションを設け、当該トピックに関連する標準的な参考文献（ドキュメント、記事、チュートリアルなど）を列挙するとともに、AIが使用された具体的なタスクとプロジェクトのどの部分で使用されたかを記載すること。

プロジェクトの内容によっては、追加のセクションが必要になる場合がある（例：使用例、機能一覧、技術的選択事項など）。

必要な追加項目がある場合は、以下に明示的に記載する。

- ・「説明」セクションには、プロジェクトの明確な名称とその主要な特徴も記載すること。
- ・「使用方法」セクションでは、必要な前提条件（ソフトウェア、ツール、バージョン、.env設定などの構成など）をすべて明記するとともに、プロジェクトを実行するための段階的な手順を記載すること。

このアクティビティで必要となる追加セクション：

- ・チーム情報：
README.mdの冒頭に記載されている各チームメンバーについて、以下の情報を提供すること：

- ・割り当てられた役割: PO（プロダクトオーナー）、PM（プロジェクトマネージャー）、技術リード、開発者など
 - 各メンバーの責任範囲の簡単な説明
- ・プロジェクト管理:
 - チームがどのように作業を組織化したか（タスクの割り当て、ミーティングの実施方法など）
 - ・プロジェクト管理に使用したツール（GitHub Issues、Trelloなど）
 - ・使用したコミュニケーションチャンネル（Discord、Slackなど）
- ・技術スタック:
 - ・使用したフロントエンド技術およびフレームワーク
 - ・使用したバックエンド技術およびフレームワーク
 - ・採用したデータベースシステムとその選定理由
 - ・その他の重要な技術やライブラリ
 - ・主要な技術的選択を行った根拠
- ・データベーススキーマ:
 - ・データベース構造の視覚的表現または説明
 - テーブル/コレクションとその関係性
 - 主キーとデータ型
- ・機能一覧:
 - ・実装済み機能の完全なリスト
 - 各機能を担当したチームメンバー
 - ・各機能の機能概要の簡潔な説明
- ・モジュール:
 - 選択したすべてのモジュールの一覧（主要モジュールおよび補助モジュール）
 - ポイント計算 (Major = 2pts（補助モジュール=1点）)
 - 特にカスタム「選択モジュール」を選択した場合の各モジュール選択理由
 - 各モジュールの実装方法
 - 各モジュールを担当したチームメンバー
- ・個人の貢献内容:
 - 各チームメンバーが具体的にどのような貢献をしたかの詳細な内訳
 - ・各メンバーが実装した具体的な機能、モジュール、またはコンポーネント
 - 直面した課題とその解決方法

その他、有用または関連する情報があれば記載可（使用マニュアル、既知の制限事項、ライセンス、クレジット情報など）



README.mdはプロジェクト評価において非常に重要な要素である。以下の要件を満たす必要がある：

- ・明確で体系的に整理されていること
- ・必要なすべてのセクションを網羅していること
- ・プロフェッショナルな体裁で、読みやすいこと

貢献内容や課題については正直に記載すること。

質の低い、あるいは不完全なREADMEは、評価に悪影響を及ぼす可能性がある。



READMEは英語で記述しなければならない。

第VII章

ボーナスパート

ボーナスパートは、最低14の必須ポイントに対応するすべての必須モジュールが実装されている場合にのみ評価される。

必要な14モジュールを超えて実装された追加モジュールは、ボーナスとして 各追加モジュール
評価される。

について:

- ・完全に機能していること
- ・モジュール要件の説明を満たしていること。プロジェクトに実質的な価値を追加する必要がある
- ・READMEに適切な正当性説明を含めること

各検証済みの追加モジュールは、

審査時に以下のように考慮される:

- ・主要モジュール: 1モジュールあたり2ポイント
- ・補助モジュール: 1モジュールあたり1ポイント

$r \ 2 \text{ major modules} + 1$

最大5ポイントまで取得可能である（例: 5つの補助モジュール、補助モジュール）。

第VIII章 提出とピア評価

通常どおりGitリポジトリに課題を提出すること。評価対象となるのはリポジトリ内の作業のみである。ファイル名を必ず再確認すること。

プロジェクト作業を開始する前に、チームメンバーや仲間とアイデアについて十分に話し合うことを強く推奨する。

評価期間中に、プロジェクトの軽微な修正が時折求められる場合がある。これには、動作のわずかな変更、OIの書き換えのための数行のコード追加、または容易に追加可能な機能などが含まれる可能性がある。

この手順がすべてのプロジェクトに適用されるわけではないが、評価ガイドラインに記載されている場合は対応できるように準備しておくこと。

なお、この手順はプロジェクトの特定部分に対する実際の理解度を確認するためのものである。修正作業は任意の開発環境（普段使用している環境など）で行うことができ、評価で特に時間枠が指定されていない限り、数分で完了できる程度のものであるべきである。

例えば、関数やスクリプトの小規模な更新、表示の変更、新しい情報を格納するためのデータ構造の調整などが課題として与えられる場合がある。

具体的な内容（範囲、対象など）は評価ガイドラインで明示され、同じプロジェクトであっても評価ごとに異なる場合がある。